OPEN-BOOK CHEATNOTE

LECTURES 9-13 + LAB 5 | PRACTICE QUIZ A-F

01 字典、循环与函数：先看输入结构，再选代码模板

L9 | Dictionary 字典

```
d = {"Amy": 8, "Ben": 6}
d["Amy"]              # value: 8
d["Amy"] += 2         # update
d["Cara"] = 7         # add new key
list(d.keys())        # names
list(d.values())      # scores
len(d)                # number of keys
for name in d:        # loops over keys
    score = d[name]
```

嵌套字典：d[name]["year"] 从左到右查两次。key 区分大小写；字符串需引号。name in d 检查 key。缺少 key 会 KeyError。

Quiz A | Running max / min

```
names = list(scores.keys())
best_name = names[0]
best = scores[best_name]
for name in names:
    value = scores[name]
    if value > best:
        best = value
        best_name = name
```

以上适用于非空字典；找最小值改为 <。同时更新名字和数值。Quiz A 指定 oldest_year=3000 时照题目写；年份越小越年长。从0开始找最值需依赖分数范围。

Quiz B | 按天跨对象求平均

```
for day in days:
    total = 0
    for city in selected:
        total += selected[city][day]
    avg = total / len(selected)
    print(f"{day}: {avg:.2f}")
```

先确定平均的方向：每天跨城市平均，外层就是 day。每组 total 归零；分母是实际对象数，不是天数。

DataFrame / random

```
df = pd.DataFrame({
    "name": ["Amy", "Ben"],
    "score": [8, 6]})
names = list(d.keys())
random.shuffle(names)  # changes list
chosen = random.sample(names, k=2)
```

DataFrame：key为列名，各列list等长。shuffle返回None，不写names=shuffle(names)。sample不重复抽取；random.choice选1个；randint(a,b)包括两端。

L10 | def / return / 参数

```
def f(x, y):
    result = x**2 + y
    return result
answer = f(2, 3)
answer = f(y=3, x=2)
```

def定义，f(...)调用。按位置看顺序，按关键词看名字。外面实参变量名不必与形参相同。函数内部赋值通常为local；可读global，传数据参数更便于复用。

return交回值并结束整个函数；print只显示。没有return默认返回None。返回局部变量的值，不会把该变量名变成global。

```
def pair(x):
    return x, x**2
both = pair(3)        # (3, 9)
a, b = pair(3)        # unpack
# ((a,b,c), d) = vector_and_scalar(...)
```

Quiz C | while 写进函数

```
def years_to_double(rate):
    assert rate > 0, "rate must be positive"
    population = 1000
    years = 0
    while population < 2000:
        population *= 1 + rate
        years += 1
    return years
for r in [0.01, 0.02, 0.05, 0.10]:
    print(years_to_double(r))
```

结果：70、36、15、8。每次调用重置起点和计数器；return放循环外。复利：P*(1+r/n)**(n*t)；1%=0.01。注意分母括号：a/(2*b)不是a/2*b。

输入参数 vs 函数内部 input

```
def ask_for_skill():
    # gets score with input() internally
    ...
# skill = ask_for_skill()
# type 6.72 in the input box
# NOT ask_for_skill(6.72)
```

需要调用者提供分数：def f(skill)。函数内部询问分数：def ask_for_skill()。外面要用分数，内部必须return skill；直接测试可写skill=6.72。

累计成功 vs 连续成功

```
while streak < target:
    outcome = np.random.choice(
        [True, False], p=[0.8, 0.2])
    trials += 1
    if outcome:
        streak += 1
    else:
        streak = 0
```

连续成功：失败归零。累计成功：失败时不归零。实验总次数trials每轮加1。

OPEN-BOOK CHEATNOTE

LECTURES 9-13 + LAB 5 | PRACTICE QUIZ A-F

02 lambda、apply、异常与输入：返回值、缩进和数据类型

L11 | lambda 与 apply

```
fn = lambda x, y: y + x**2 + 2*x + 1
both = lambda x,y: (
    y+x**2+2*x+1, (2*x+2, 1))
value, gradient = both(2, 1)
# value=10; gradient=(6, 1)
```

lambda只有一个表达式，自动返回，不写return。多结果用tuple括号。

```
def grade_status(numeric_grade):
    if numeric_grade >= 55:
        return "status = pass"
    else:
        return "status = fail"
result = df["numeric_grade"].apply(
    grade_status)
display(result)
# pass function, NOT grade_status()
```

apply把列中每个值分别传入函数，产生新Series。保存新列：df["status"]=result。Series.name="label"改名称。round(x,2)改数值；f"{x:.2f}"改显示格式。

无返回值的绘图模板

```
def histogram(values, xlabel, title):
    plt.figure()
    plt.hist(values, bins=20)
    plt.xlabel(xlabel)
    plt.ylabel("Frequency")
    plt.title(title)
    plt.show()
# bar: plt.bar(categories, heights)
```

hist统计数值分布；bar给定类别和柱高。绘图函数可以只显示图，不需要return；返回值仍为None。

Mutation：原地修改 vs 新对象

```
def change(lst):
    lst.append("New!")  # original changes

def rebind(lst):
    lst = lst + ["New!"] # local new list

def new_list(lst):
    return lst + ["New!"]
y = new_list([1, 2, 3])
```

lst[-1]=...、append修改原对象；局部lst=新列表不会让外面的名字改指向。想保留新对象，return并接收。整数NumPy array赋值0.1会变0；用dtype=float可保留小数。

L12 | 错误类型速查

SyntaxError / IndentationError：冒号、括号、缩进。
NameError：未定义名字/局部变量。
KeyError：字典key不存在。
IndexError：list索引越界。
TypeError：类型不支持，如"3"+2。
ValueError：值无法转换，如int("many")。
ZeroDivisionError：普通数值除以0。

Traceback先看底部错误类型，再看出错行和调用链。Warning通常不停止；NumPy无效除法可能warning+nan，而非ZeroDivisionError。

Quiz E | try / except 订单模板

```
def order_total(order, prices):
    total = 0.0
    for item, quantity in order.items():
        try:
            total += prices[item]*int(quantity)
        except KeyError:
            print(f"We do not sell {item}")
        except ValueError:
            print(f"Invalid quantity: {quantity}")
    return total
```

不要先if item in prices：缺商品会直接跳过，无法进入except打印。order是想买什么，prices才是店里卖什么。失败后跳过try剩余代码。此模板except后无其他循环操作，不需要continue。

input验证：转换 + 范围

```
def ask_for_skill():
    while True:
        typed = input("Score (0-10): ")
        try:
            skill = float(typed)
        except ValueError:
            print("Enter a number")
            continue
        if 0 <= skill <= 10:
            return skill
        print("Out of range")
# skill = ask_for_skill()
```

input总是str；整数用int，小数用float。int("3.5")失败；int("-3")成功但可能超范围。continue下一轮；break退出当前循环；return退出整个函数。

空组与异常处理原则

求平均前：if count==0: return None，不要除以0。expected错误用具体except；避免except: pass无声丢数据。打印跳过项或记录num_skipped。捕获错误不会自动修复数据。

OPEN-BOOK CHEATNOTE

LECTURES 9-13 + LAB 5 | PRACTICE QUIZ A-F

03 assert、debugging 与 Lab 5：边界测试和关键词参数

L13 | assert 与 tests

```
assert 0 <= p <= 1, "Invalid probability"
assert 0 <= N < len(scores), "Invalid N"
assert len(scores) == expected_number, (
    "The roster is not the expected size")

beats = lambda skill, b: skill > b
assert beats(7.94, 6.72) == True
assert beats(1.51, 6.72) == False
assert beats(6.72, 6.72) == False
```

真：不输出，继续；假：AssertionError。逗号后是失败消息。Precondition在函数开始；postcondition在返回前。try/except+if用于用户输入重试；assert用于检查程序/数据假设。相等案例可发现>被误写成>=。

缺失编码 -999 与清洗

```
clean = {}
dropped = 0
for name in updated:
    score = updated[name]
    if score >= 0:
        clean[name] = score
    else:
        dropped += 1
print("Kept", len(clean), "Dropped", dropped)
```

-999是这份数据的缺失标记，不是Python特殊值。dropped=没复制到新字典；原字典不变。检查人数、数字类型与0-10范围；清洗后47人，不能要求50。0若作为缺失编码，范围检查发现不了。删行会改变样本构成。

Lab partner 策略骨架

```
order = list(scores.keys())
random.shuffle(order)
benchmark = 0
for i in range(N):
    benchmark = max(benchmark, scores[order[i]])
chosen = order[-1]  # fallback
for i in range(N, len(order)):
    if scores[order[i]] > benchmark:
        chosen = order[i]
        break
```

前N人只设标准；剩下第一个严格超过者即选。无人超过则选最后一个，-999可能由此进入结果。缺break会变最后一个合格者。N=0且所有分数>0时应选位置0；可返回位置进行测试。

生日 / silent-error 检查清单

360天模型：j in range(i+1,n)，避免自己与自己比较。1人必无重复，361人必重复。成功比例=成功次数/固定实验次数。每组count/total归零；人数累计用+=1，不是=1。"9.99">"10.50"为True，比较数值前先float。平均值须在最小值与最大值间；正确范围不保证逻辑正确。debug：缩小例子、打印中间值、一次改一处、修复后保留测试。

Lab 5 | 三种星号：不要混淆

```
def f(a, b, *, option=False):
    ...  # option is keyword-only
f(1, 2, option=True)
# f(1, 2, True) -> TypeError

def all_named(*, x, y):
    ...  # all must be named

def collect(*args, **kwargs):
    return args, kwargs
# collect(2, 4, color="blue")
# -> ((2, 4), {"color": "blue"})
```

单独*：后面必须用名字传入。
*args：额外位置参数收集为tuple；无输入为()。
**kwargs：额外关键词参数收集为dict；无输入为{}。args/kwargs是惯例名字，不是关键词。默认值≠keyword-only；name=None可省略，*,name=None还限制传入方式。

```
def show_fourth(*args):
    if len(args) >= 4:
        print(args[3])
    else:
        print("Not enough arguments")

# kwargs: check key before lookup
# if "color" in kwargs:
#     color = kwargs["color"]

print("Port", "au", "Prince, Haiti", sep="-")
```

位置参数先于显式关键词：f(x=1,2)是SyntaxError。关键词可换顺序，但同一参数不可重复赋值。调用f(*values)展开序列，f(**options)展开字典，与定义时的收集相反。sep是分隔设置，不是待打印项。

Quiz F | 默认选项 + 两点距离

```
def point_distance(p1, p2, *, manhattan=False):
    x1, y1 = p1
    x2, y2 = p2
    if manhattan:
        return abs(x1-x2) + abs(y1-y2)
    return np.sqrt((x1-x2)**2 + (y1-y2)**2)

assert point_distance((0,0),(3,4)) == 5
assert point_distance(
    (0,0),(3,4),manhattan=True) == 7
```

if manhattan表示True时用Manhattan；别把两种算法写反。同点距离为0只能测部分情况，不能区分两种公式。

原Quiz F | polar坐标转换

```
def cord_dist(p1, p2, *, polar=False):
    if polar:
        r1, t1 = p1
        r2, t2 = p2
        p1 = (r1*np.cos(t1), r1*np.sin(t1))
        p2 = (r2*np.cos(t2), r2*np.sin(t2))
    return euclidean_dist(p1, p2)
```

polar=True：输入(r,theta)，先转(x,y)。角度用弧度，90度=np.pi/2。return放if外，False时也有结果。polar是参数名，不是Python关键词。